

Author: Yanis Mignot
Student ID: H00550788
Date: October 2026

Technical Documentation & Developer Guide:

Yahtzee C# .NET



Course: F20SC - Industrial Programming
Project: CW1 — Yahtzee Game Implementation
Language & Platform: C# 13 / .NET 10.0
GUI Framework: Avalonia UI
Database: SQLite

1. Requirements Checklist

The following feature checklist is based on the MoSCoW prioritization model outlined in Appendix A of the coursework specification. All basic and advanced functional requirements have been fully implemented and verified.

FEATURE	STATUS	IMPLEMENTATION NOTES
Single-player gameplay	Fully implemented	Solo game: supports 1 player competing against a personal scorecard across 13 rounds.
Multi-player gameplay	Fully implemented	Hotseat multiplayer: supports 2 to 4 human players sharing the same screen with turn-by-turn notification.
Specifying player names	Fully implemented	Custom name entry input for all players before launching a session.
Displaying whose turn is it	Fully implemented	Top header status bar displays current player turn name, round count, and player active tab indicators.
Displaying the number of rolls available	Fully implemented	Visual badge and action button count down remaining rolls (3→2→1→0) and disable rolling when exhausted.
Indicating which dice will be rolled/set aside	Fully implemented	Interactive dice control cards highlight held dice with green badges and border accents.
Score preview	Fully implemented	Live score calculation previews point gains for all 13 categories after every roll before committing.
Saving/loading games	Fully implemented	Complete game state serialization to JSON format allowing save/resume at any roll or round.
Offer to save the game before exiting	Fully implemented	Interception of exit navigation and window closing events with confirmation modal prompting save option.
Result summary at the end of the game	Fully implemented	Modal display of ranked final leaderboard showing winner crowning, scores, and standings.

Games database	Fully implemented	SQLite embedded database recording finished games, top 5 overall scores, player high scores, and match history.
AI opponents	Fully implemented	Multi-AI support with selectable strategies : Basic AI heuristic and Smart AI tactical combination evaluation.

2. Design Considerations

2.1 Architecture & Design Patterns

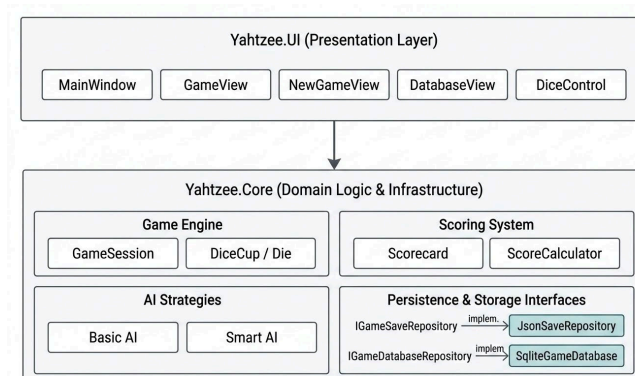
The application follows a Decoupled Layered Architecture adhering to SOLID object-oriented principles. The codebase is organized into two primary projects

Yahtzee.Core (Domain Logic & Infrastructure):

- Contains pure C# domain entities, scoring calculators, AI decision engines, and storage abstractions.
- Completely decoupled from UI frameworks, enabling full portability, unit testing, and headless execution.

Yahtzee.UI (Presentation Layer):

- Built using Avalonia UI, handling visual controls, user input events, and responsive layout styling.



Key Design Patterns Implemented:

- Strategy Pattern (*IAiStrategy*): Encapsulates computer decision-making algorithms, allowing easy extension of new AI behavior models without modifying game engine logic.
- Repository Pattern (*IGameSaveRepository*, *IGameDatabaseRepository*): Abstracts data persistence mechanisms (file I/O and SQLite transactions), facilitating mock testing and storage decoupling.
- Dependency Injection / Dependency Inversion: Service interfaces like *IRandomProvider* allow injecting deterministic pseudo-random generators for unit tests.

2.2 GUI Framework Choice: Justification of Avalonia UI

While traditional Windows Forms was suggested in the specification as a basic Windows option, Avalonia UI (.NET 10 XAML) was selected for the following technical reasons:

True Cross-Platform Capability:

Windows Forms is legacy and Windows-exclusive. Avalonia UI compiles and runs natively across Windows 11, GNU/Linux, and macOS with identical visual behavior.

Modern XAML & Data Binding:

Provides declarative UI layout via XAML, styling dictionaries (*Colors.xaml*), custom control template encapsulation (*DiceControl.xaml*), and flexible layout grids.

Prior Experience with the Framework:

Avalonia UI was chosen because it is a library the author has extensive prior experience with, allowing for faster development and more reliable implementation compared to other unfamiliar UI frameworks.

2.3 Choice of Data Structures

`ReadOnlyList<T>` & `List<T>`:

Used for ordered sequences such as player turns and dice collections where index access is needed.

`Dictionary<ScoreCategory, int?>`:

Serves as the backing store for Scorecard, mapping each enum category key to an optional integer score (null represents unfilled categories, 0 represents a scratched category).

HashSet<int>:

Employed by DiceCup and IAiStrategy to hold indices of dice marked for retention (0 to 4), ensuring :

- O(1)
- O(1) lookups, insertions, and uniqueness.
- *IEnumerable<T>* with LINQ: Utilized extensively throughout ScoreCalculator and SmartAiStrategy for functional queries (LINQ method: Fluent Syntax) (GroupBy, Select, OrderByDescending).3. Developer Guide

3.1 Code Organization & Class Responsibilities

Yahtzee.Core.Models

Die: Represents a single six-sided die. Tracks current face value (1–6) and held status (IsHeld).

DiceCup: Manages a set of 5 Die objects. Controls rolling logic, tracking RollsRemaining (max 3), toggle hold states, and round resets.

Player (*Abstract Base Class*): Base model for game participants. Properties include Id, Name, and an instantiated Scorecard.

HumanPlayer: Derived class representing human users.

AiPlayer: Derived class representing bot players, maintaining an associated StrategyName.

Yahtzee.Core.Scoring

ScoreCategory (*Enum*): Defines the 13 official categories (Aces, Twos, Threes, Fours, Fives, Sixes, ThreeOfAKind, FourOfAKind, FullHouse, SmallStraight, LargeStraight, Yahtzee, Chance).

Scorecard: Tracks filled vs. open categories. Calculates section subtotals, upper section bonus (35 points unlocked at

- ≥ 63

- ≥ 63 points subtotal), lower section total, and overall total score.

ScoreCalculator: Pure static utility evaluating dice values against category scoring rules and generating score previews for all categories.

Yahtzee.Core.Game

GameSession: Central state machine controller. Manages 13 game rounds, player turn progression, roll validation, AI turn execution orchestration, game-over state evaluation, and save/restore data mapping.

Yahtzee.Core.AI

IAiStrategy: Contract interface defining methods ChooseDiceToHold(...) and ChooseCategoryToFill(...).

BasicAiStrategy: Heuristic AI holding dice with matching values or highest die values.

SmartAiStrategy: Advanced tactical AI evaluating category probabilities, straight sequences, full house opportunities, and optimal category scoring priorities.

Yahtzee.Core.Persistence & Yahtzee.Core.Database

JsonSaveRepository: Implements IGameSaveRepository using System.Text.Json to serialize and deserialize active GameSaveData snapshots.

SqliteGameDatabase: Implements IGameDatabaseRepository using ADO.NET (Microsoft.Data.Sqlite) for connection management, schema initialization, recording match results, and executing leaderboard queries.

3.2 Key Class Interfaces & Encapsulation

```
namespace Yahtzee.Core.AI
{
    public interface IAiStrategy
    {
        string Name { get; }
        ISet<int> ChooseDiceToHold(IReadOnlyList<int> diceValues, Scorecard scorecard, int rollsRemaining);
        ScoreCategory ChooseCategoryToFill(IReadOnlyList<int> diceValues, Scorecard scorecard);
    }
}

namespace Yahtzee.Core.Persistence
{
    public interface IGameSaveRepository
    {
        Task SaveGameAsync(GameSaveData saveData, string filePath);
        Task<GameSaveData> LoadGameAsync(string filePath);
        IEnumerable<string> ListSaveFiles(string directoryPath);
    }
}

namespace Yahtzee.Core.Database
{
    public interface IGameDatabaseRepository
    {
        Task InitializeDatabaseAsync();
        Task RecordCompletedGameAsync(DateTime playedAt, IEnumerable<(string name, int score, bool isAi)> playerScores);
        Task<List<HighScoreRecord>> GetTop5HighScoresOverallAsync();
        Task<List<HighScoreRecord>> GetTop5HighScoresForPlayerAsync(string playerName);
        Task<List<CompletedGameRecord>> GetRecentGamesAsync(int limit = 20);
    }
}
```

3.3 Save-File Format Specification (.json)

Saved games are stored in JSON format (*GameSaveData*). This format captures the complete game state, enabling accurate restoration of mid-turn dice values, rolls remaining, and player scorecards.

JSON Schema Specification & Example:

```
{
  "savedAt": "2026-10-08T14:30:00.1234567Z",
  "currentRound": 5,
  "currentPlayerIndex": 0,
  "rollsRemaining": 2,
  "diceValues": [5, 5, 5, 2, 6],
  "diceHolds": [true, true, true, false, false],
  "players": [
    {
      "name": "Alice",
      "isAi": false,
      "strategyName": null,
      "scores": {
        "Aces": 3,
        "Twos": null,
        "Threes": 9,
        "Fours": null,
        "Fives": 15,
        "Sixes": null,
        "ThreeOfAKind": null,
        "FourOfAKind": null,
        "FullHouse": null,
        "SmallStraight": null,
        "LargeStraight": null,
        "Yahtzee": null,
        "Chance": null
      }
    },
    {
      "name": "Computer (Smart)",
      "isAi": true,
      "strategyName": "Smart AI",
      "scores": {
        "Aces": 4,
        "Twos": 6,
        "Threes": null,
        "Fours": 16,
        "Fives": null,
        "Sixes": null,
        "ThreeOfAKind": null,
        "FourOfAKind": null,
        "FullHouse": 25,
        "SmallStraight": null,
        "LargeStraight": null,
        "Yahtzee": null,
        "Chance": null
      }
    }
  ]
}
```

3.4 Games Database Structure (SQLite)

The application uses an embedded SQLite database (yahtzee_games.db) managed via *SqlitYahtzee.Core.Persistence* & *Yahtzee.Core.DatabaseGameDatabase.cs*

Relational Database:

CompletedGames

Champ	Type de données	Contraintes / Notes
Id ⁽¹⁾	TEXT	PRIMARY KEY
PlayedAt	TEXT	NOT NULL
PlayerCount	INTEGER	NOT NULL
WinnerName	TEXT	NOT NULL
WinnerScore	INTEGER	NOT NULL

GameScores

Champ	Type de données	Contraintes / Notes
Id	TEXT	PRIMARY KEY
GameId ⁽¹⁾	TEXT	NOT NULL
PlayerName	INTEGER	NOT NULL
Score	TEXT	NOT NULL
IsAi	INTEGER	NOT NULL
PlayedAt	TEXT	NOT NULL

(1) : Foreign Key

DDL Statements:

```
CREATE TABLE IF NOT EXISTS CompletedGames (  
    Id TEXT PRIMARY KEY,  
    PlayedAt TEXT NOT NULL,  
    PlayerCount INTEGER NOT NULL,  
    WinnerName TEXT NOT NULL,  
    WinnerScore INTEGER NOT NULL  
);  
  
CREATE TABLE IF NOT EXISTS GameScores (  
    Id TEXT PRIMARY KEY,  
    GameId TEXT NOT NULL,  
    PlayerName TEXT NOT NULL,  
    Score INTEGER NOT NULL,  
    IsAi INTEGER NOT NULL,  
    PlayedAt TEXT NOT NULL,  
    FOREIGN KEY(GameId) REFERENCES CompletedGames(Id)  
);
```

```
CREATE INDEX IF NOT EXISTS IX_GameScores_Score ON GameScores(Score DESC);  
CREATE INDEX IF NOT EXISTS IX_GameScores_PlayerName ON GameScores(PlayerName);
```

High Score & Leaderboard SQL Queries:

Top 5 Overall High Scores:

```
SELECT PlayerName, Score, IsAi, GameId, PlayedAt  
FROM GameScores  
ORDER BY Score DESC  
LIMIT 5;
```

Top 5 High Scores for a Specific Player:

```
SELECT PlayerName, Score, IsAi, GameId, PlayedAt  
FROM GameScores  
WHERE PlayerName = @PlayerName  
ORDER BY Score DESC  
LIMIT 5;
```

4. Conclusion & Reflection

The Yahtzee C# application satisfies all functional requirements specified in the coursework remit. By leveraging modern .NET 10 software design technique, including cross-platform GUI architecture, strict separation of concerns, polymorphic AI strategies, JSON persistence, and transactional SQLite database storage, the project delivers a robust, extensible, and high-performance gaming application.